```
      })
      .willRespondWith({
        status: 409,
        body: {
          error: string(`Movie ${name} already exists`)
        }
      });

    await provider.executeTest(async mockProvider => {
      const movies = await addNewMovie(mockProvider.url, name, year);
      expect(movies.error).toEqual(`Movie ${name} already exists`);
    });
  });
});

describe('When a DELETE request is made to /movies', () => {
  test('it should throw an error if movie to delete does not exist', async () =>
{
    const testId = 643256;

    provider
      .uponReceiving('a request to delete a movie that does not exists')
      .withRequest({
        method: 'DELETE',
        path: `/movie/${testId}`,
      })
      .willRespondWith({
        status: 404,
        body: {
          error: string(`Movie ${testId} not found`)
        }
      });

    await provider.executeTest(async mockProvider => {
      const movies = await deleteMovie(mockProvider.url, testId);
      expect(movies.error).toEqual(`Movie ${testId} not found`);
    });
  });

  test('it should delete an existing movie successfully', async () => {
    const testId = 100;
    EXPECTED_BODY.id = testId;

    provider
      .given('there is a movie with specific ID', { id: testId })
      .uponReceiving('a request to delete a movie that exists')
      .withRequest({
        method: 'DELETE',
        path: `/movie/${testId}`,
      })
      .willRespondWith({
        status: 200,
        body: {
          message: string(`Movie ${testId} has been deleted`)
        }
      });

    await provider.executeTest(async mockProvider => {
      const movies = await deleteMovie(mockProvider.url, testId);
      expect(movies.message).toEqual(`Movie ${testId} has been deleted`);
    });
```